



PUC Minas

Aulão 1

AULÃO DE AEDS 2



```
adystatechan
String Function Ar
on F(e){var t=_[e]={},
&&e.stopPropagation(){r=!1;break
ch:r&&(s=t,c(r))}return thi
return u=[],this},disable:fu
ion(){return p.fireWith(this,a
r={state:function(){return n},
e.promise().done(n.resolve).fa
on(){n=s},t[1^e][2].disable,
ll(arguments),r=n.length,i=
v(r);r>t;t++)n[t]&&b.isFu
able><a href='/a'>a</a
t")[0],r.style.css
te("style"))
```



PUC Minas

SOMATÓRIOS



Qual o método de resolução de somatórios?

A ideia de resolver um somatório é basicamente quebrar ele em somatórios mais simples, dos quais já conhecemos a fórmula fechada. Assim substituímos o símbolo do somatório e ficamos com apenas equações. **O que veremos a seguir são técnicas para simplificar os somatórios.**

Regras básicas de manipulação

Distributividade

Se eu tenho uma constante eu posso jogar pra fora ou pra dentro! Isso vale tanto para divisões tanto para multiplicações.

$$\sum_{i \in I} c \cdot a_i = c \cdot \sum_{i \in I} a_i$$

$$\sum_{i \in I} \frac{a_i}{c} = \frac{1}{c} \cdot \sum_{i \in I} a_i$$

Associatividade

É possível quebrar somas e subtrações em diferentes somatórios. No entanto, não se pode fazer o mesmo com multiplicações e divisões

$$\sum_{i \in I} (a_i + b_i) = \sum_{i \in I} a_i + \sum_{i \in I} b_i$$

$$\sum_{i \in I} (a_i - b_i) = \sum_{i \in I} a_i - \sum_{i \in I} b_i$$

Comutatividade

Não importa se eu to fazendo um for somando ou decrementando, se os limites forem os mesmos (apesar de invertidos) o custo vai ser o mesmo.

// Do início ao fim
`for(i = 0; i < n; i++)`

// É idêntico ao reverso
`for(i = n-1; i >= 0; i--)`

Propriedades da manipulação

P1 - Combinação de Conjuntos

Com algumas adaptações é possível unificar somatórios com limites diferentes.

$$\sum_1^m a_i + \sum_m^n a_i = \sum_1^n a_i + a_m$$

P2 - Base pra Perturbação

Ao usar a propriedade da perturbação para obter o somatório de uma potência, devemos sempre perturbar o somatório de uma potência acima, por causa do seguinte:

$$S_n + a_{n+1} = a_0 + \sum_{0 \leq i \leq n} a_{i+1}$$

Prova por Indução

Primeiro testamos o **caso base**
(o primeiro caso para qual o limite superior é válido).

Se o caso base estiver correto partimos para a fórmula.

Nesse caso o n seria o limite superior e o $n - 1$ é sempre usado como $n - 1$.

$$S_n = S_{n-1} + a_n$$

Do Código ao Somatório

Análise de Custo

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= 3*i + 2; j++) {  
        x++;  
    }  
}
```

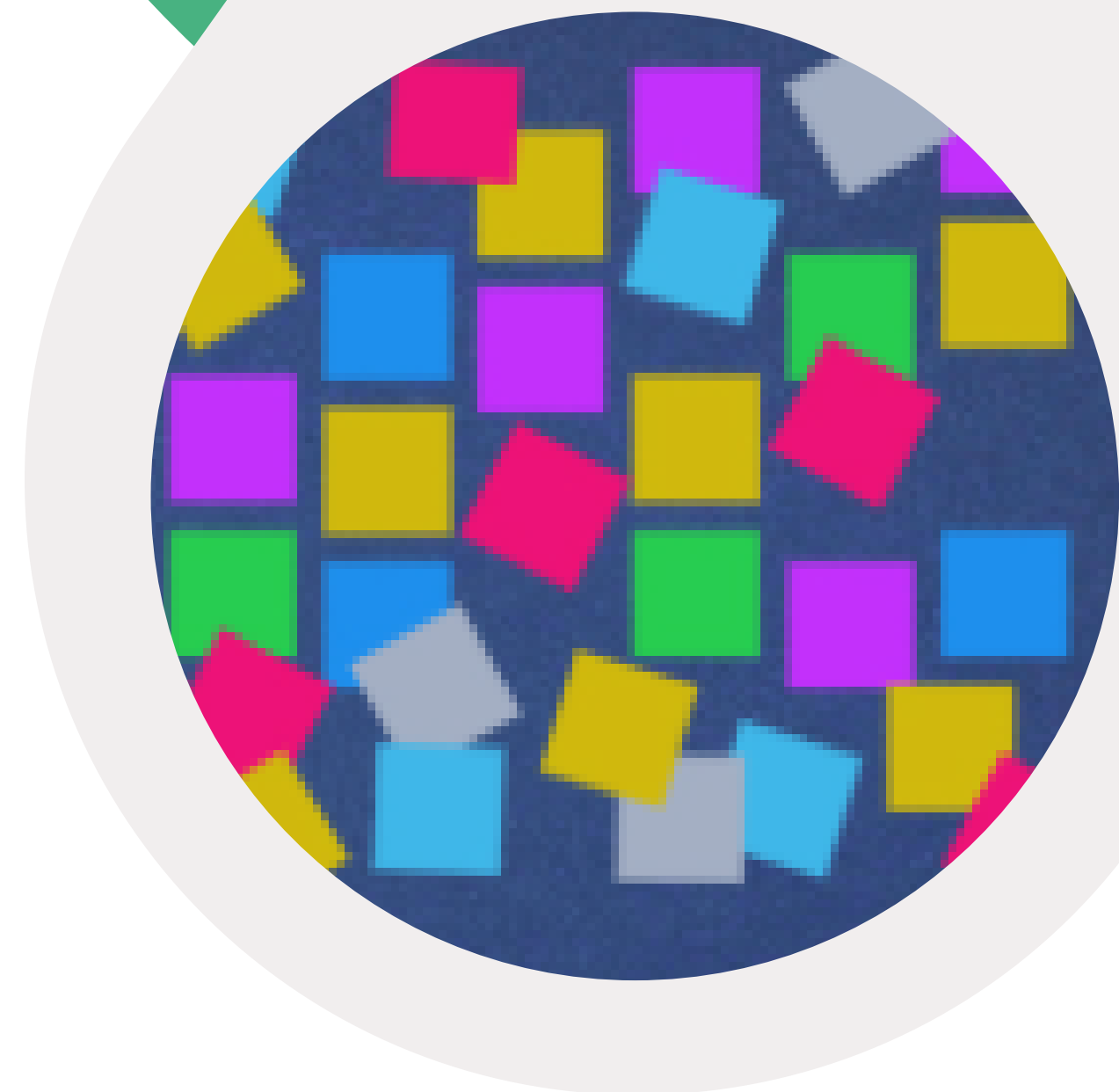
O que observar?

- Laço externo: varia com $i = 1$ até n .
- Laço interno: executa exatamente $(3i + 2)$ vezes para cada i .
- Operação básica: $x++$ tem custo constante ($c = 1$).



PUC Minas

Ordenação



Seleção

Varre o resto do vetor, escolhe o menor elemento e troca de lugar direto com a base.

```
1 Receba um vetor com n elementos.
2 Para i = 0 até n-2 faça:
3     Inicie min = i
4     Para j = i+1 até n-1 faça:
5         Se vetor[j] < vetor[min], então min = j
6     fim_para
7     Se min ≠ i, então
8         troque vetor[i] com vetor[min]
9     fim_se
10 fim_para
11 Retorne o vetor ordenado.
```

Custo: Sempre $\Theta(n^2)$
(independe da ordem)

Inserção

Como organizar cartas de baralho na mão: pega a próxima e desloca as maiores para abrir espaço.

```
1 Receba um vetor com n elementos.
2 Para i = 1 até n-1 faça:
3     atual = vetor[i]
4     j = i - 1
5     Enquanto j >= 0 e vetor[j] > atual faça:
6         vetor[j + 1] = vetor[j]
7         j = j - 1
8     fim-enquanto
9     vetor[j + 1] = atual
10 fim-para
11 Retorne o vetor ordenado.
```

Custo: Melhor $\Omega(n)$
Pior $O(n^2)$

Bolha

Bolhas de ar na água: compara vizinhos dois a dois e empurra o maior para o final da fila a cada rodada.

```
1 Receba um vetor com n elementos.
2 Para i = n - 1 até 1 passo -1 faça:
3     Para j = 0 até i - 1 faça:
4         Se vetor[j] > vetor[j + 1] então:
5             troque vetor[j] com vetor[j + 1]
6         fim_se
7     fim_para
8 fim_para
9 Retorne o vetor ordenado.
```

Custo: Melhor $\Omega(n)^*$
Pior $O(n^2)$

*Com flag de parada

Quicksort

Escolhe um pivô de referência e particiona o vetor em dois lados: menores à esquerda e maiores à direita. Ao cruzar os ponteiros, o pivô já delimitou as metades que serão ordenadas recursivamente.

```
1 Funcao quicksort(esq, dir):
2   i = esq, j = dir
3   // Passo 1: Escolha do pivo central
4   pivo = vetor[(esq + dir) / 2]
5   // Passo 2: Particao em torno do pivo
6   Enquanto i <= j faca:
7     Enquanto vetor[i] < pivo faca: i = i + 1
8     Enquanto vetor[j] > pivo faca: j = j - 1
9     Se i <= j entao:
10      troque vetor[i] com vetor[j]
11      i = i + 1
12      j = j - 1
13   fim_se
14   fim_enquanto
15   // Passo 3: Chamadas recursivas para as sublistas
16   Se esq < j entao: quicksort(esq, j)
17   Se i < dir entao: quicksort(i, dir)
```

Custo: Medio $\Theta(n \log n)$
Pior $O(n^2)$

Shellsort

Extensão direta da Inserção: em vez de comparar apenas vizinhos imediatos, o Shellsort compara elementos distanciados por saltos h . Conforme o salto diminui até chegar em $h = 1$, o vetor fica quase ordenado, onde a inserção direta roda em tempo quase linear.

```
1 Receba um vetor com n elementos.
2 // Passo 1: Calcular o maior salto inicial (sequencia de Knuth)
3 h = 1
4 Repita:
5   h = (h * 3) + 1
6 Ate que h >= n
7 // Passo 2: Reduzir os intervalos e ordenar por cor
8 Repita:
9   h = h / 3
10  Para cor = 0 ate h - 1 faca:
11    // Passo 3: Insercao nos elementos distanciados por h
12    Para i = h + cor ate n - 1 passo h faca:
13      tmp = vetor[i]
14      j = i - h
15      Enquanto j >= 0 e vetor[j] > tmp faca:
16        vetor[j + h] = vetor[j]
17        j = j - h
18      fim_enquanto
19      vetor[j + h] = tmp
20    fim_para
21  fim_para
22 Ate que h == 1
23 Retorne o vetor ordenado.
```

Custo: Melhor $\Omega(n)$
Pior $O(n^{1.5})$

Countingsort

O Counting Sort não faz comparações entre elementos; ele apenas conta quantas vezes cada número apareceu (como urnas de votação) e calcula direto o índice final de cada um.

```
1 Receba um vetor com n elementos.
2 maior = descubra o maior elemento de vetor
3 Crie vetor count de tamanho (maior + 1) com zeros
4 Crie vetor ordenado de tamanho n
5
6 // Passo 1: Contar a frequência de cada elemento
7 Para i = 0 até n - 1 faça:
8     count[vetor[i]] = count[vetor[i]] + 1
9 fim_para
10
11 // Passo 2: Transformar em contagem acumulada
12 Para i = 1 até maior faça:
13     count[i] = count[i] + count[i - 1]
14 fim_para
15
16 // Passo 3: Posicionar no vetor auxiliar (estável, do fim ao início)
17 Para i = n - 1 até 0 passo -1 faça:
18     pos = count[vetor[i]] - 1
19     ordenado[pos] = vetor[i]
20     count[vetor[i]] = count[vetor[i]] - 1
21 fim_para
22
23 // Passo 4: Copiar de volta para o vetor original
24 Para i = 0 até n - 1 faça:
25     vetor[i] = ordenado[i]
26 fim_para
27
28 Retorne o vetor ordenado.
```

Custo: $\Theta(n + k)$ (k = maior valor do vetor)

Radix Sort

O Radix Sort divide para conquistar pelos dígitos: ele ordena os números dígito a dígito (da unidade até a casa mais significativa) aplicando o Counting Sort estável a cada passada.

```
1 Receba um vetor com n elementos.
2 maior = descubra o maior elemento de vetor
3 Para exp = 1 até (maior / exp > 0) passo exp = exp * 10 faça:
4     Crie vetor count de tamanho 10 inicializado com zeros
5     Crie vetor output de tamanho n
6     Para i = 0 até n - 1 faça:
7         digito = (vetor[i] / exp) mod 10
8         count[digito] = count[digito] + 1
9     fim_para
10    Para i = 1 até 9 faça:
11        count[i] = count[i] + count[i - 1]
12    fim_para
13    Para i = n - 1 até 0 passo -1 faça:
14        digito = (vetor[i] / exp) mod 10
15        pos = count[digito] - 1
16        output[pos] = vetor[i]
17        count[digito] = count[digito] - 1
18    fim_para
19    Para i = 0 até n - 1 faça:
20        vetor[i] = output[i]
21    fim_para
22 fim_para
23 Retorne o vetor ordenado.
```

Custo: $\Theta(d \cdot (n + b))$ (d = no. de dígitos, b = base 10)

Heapsort

O Heapsort transforma o vetor em uma árvore binária quase completa onde o pai é sempre maior que os filhos (Heap Máximo). A raiz sempre tem o maior elemento; basta trocá-la com o fim do vetor e restaurar a propriedade descendo o elemento substituto.

```
1  Receba um vetor com n elementos.
2  Reajuste o vetor para iniciar no índice 1 ate n
3  // Passo 1: Construir o Heap Maximo
4  Para tamHeap = 2 ate n faca:
5      i = tamHeap
6      Enquanto i > 1 e vetor[i] > vetor[i / 2] faca:
7          troque vetor[i] com vetor[i / 2]
8          i = i / 2
9      fim_enquanto
10 fim_para
11 // Passo 2: Extrair maximos e restaurar a arvore
12 tamHeap = n
13 Enquanto tamHeap > 1 faca:
14     troque vetor[1] com vetor[tamHeap]
15     tamHeap = tamHeap - 1
16     i = 1
17     Enquanto i <= tamHeap / 2 faca:
18         filho = maior filho entre 2*i e 2*i + 1
19         Se vetor[i] < vetor[filho] entao:
20             troque vetor[i] com vetor[filho]
21             i = filho
22     Senao:
23         interrompa o laco
24     fim_se
25 fim_enquanto
26 fim_enquanto
27 // Passo 3: Ajustar indices de volta
28 Reajuste o vetor de volta para indice 0 ate n-1
29 Retorne o vetor ordenado.
```

Custo: Sempre $\Theta(n \log n)$

Mergesort

Dividir para conquistar: divide o vetor ao meio recursivamente até ter pedaços unitários e depois intercala as metades já ordenadas usando sentinelas no fim para simplificar as comparações.

```
1  Funcao mergesort(esq, dir):
2      // Passo 1: Divisao recursiva
3      Se esq < dir entao:
4          meio = (esq + dir) / 2
5          mergesort(esq, meio)
6          mergesort(meio + 1, dir)
7          intercalar(esq, meio, dir)
8      fim_se
9  Funcao intercalar(esq, meio, dir):
10     // Passo 2: Criar vetores auxiliares
11     n1 = meio - esq + 1
12     n2 = dir - meio
13     Crie vetor a1 de tamanho n1 + 1 com sentinela infinito no fim
14     Crie vetor a2 de tamanho n2 + 1 com sentinela infinito no fim
15     // Passo 3: Copiar elementos
16     Copie elementos de vetor[esq .. meio] para a1
17     Copie elementos de vetor[meio+1 .. dir] para a2
18     // Passo 4: Intercalacao ordenada
19     i = 0, j = 0
20     Para k = esq ate dir faca:
21         Se a1[i] <= a2[j] entao:
22             vetor[k] = a1[i]
23             i = i + 1
24         Senao:
25             vetor[k] = a2[j]
26             j = j + 1
27     fim_se
28 fim_para
```

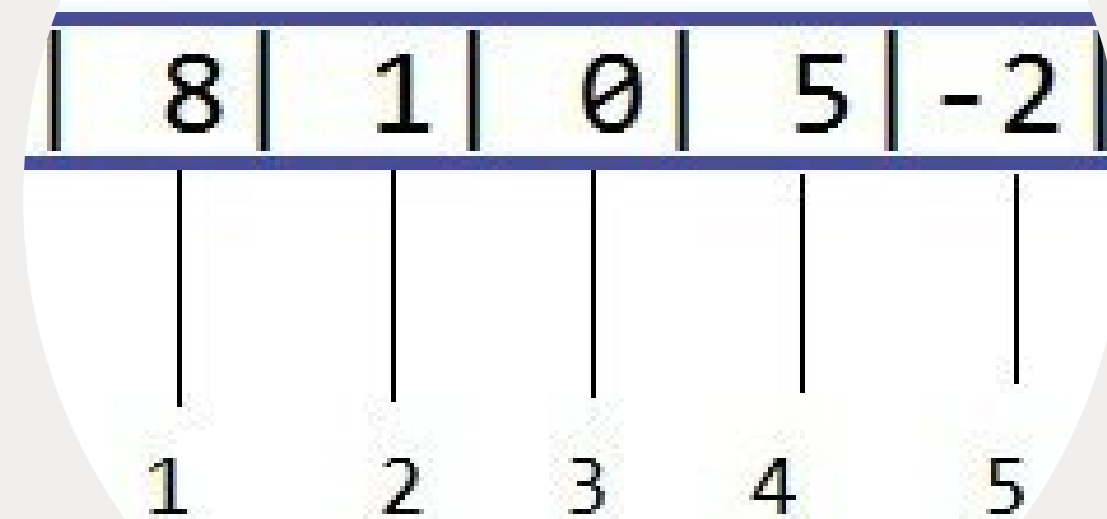
Custo: Sempre $\Theta(n \log n)$



PUC Minas

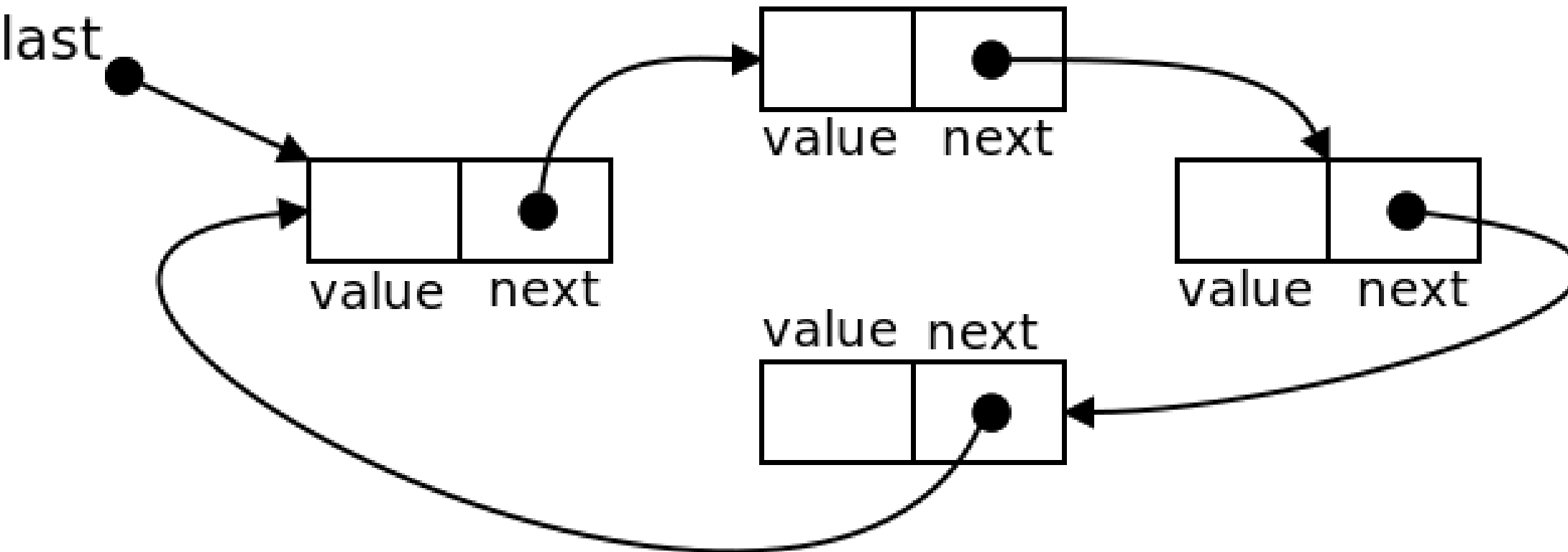
TADs Lineares

AULÃO DE AEDS 2



O que é um TAD Linear?

- TADs Lineares, ou Tipos Abstratos de Dados Lineares, são formas estruturais de organizar um conjunto de dados.
- Os estudados por vocês até então são Pilha, Lista e Fila (Simples e Circular, Simplesmente e Duplamente Encadeadas). Vamos nos aprofundar nas 3.



Fila Simples

Inserir no Fim

Em uma fila linear simples, enfileirar (inserir) é colocar o elemento no final da fila. Como n representa a quantidade atual de itens e também o próximo índice livre, a operação vai direto no endereço `array[n]` sem precisar deslocar ninguém.

```
1 Funcao enfileirar(x):
2     // Passo 1: Validar se a fila linear
3     esta cheia
4     Se n >= tamanho entao:
5         Dispare erro "Erro ao inserir!"
6     fim_se
7     // Passo 2: Inserir no fim da fila e
8     incrementar contador
9     array[n] = x
10    n = n + 1
```

Custo: Sempre $\Theta(1)$

Remover no Inicio

Essa é a operação padrão de desenfileirar (FIFO) em uma fila linear baseada em array. Como o elemento removido sempre sai do índice 0, todos os elementos restantes precisam ser deslocados uma posição para a esquerda para cobrir o espaço vazio, tornando a operação linear.

```
1 Funcao desenfileirar():
2     // Passo 1: Validar se a fila esta vazia
3     Se n == 0 entao:
4         Dispare erro "Erro ao remover!"
5     fim_se
6     // Passo 2: Salvar o primeiro elemento
7     e decrementar tamanho
8     resp = array[0]
9     n = n - 1
10    // Passo 3: Deslocar todos os
11    elementos para a esquerda
12    Para i = 0 ate n - 1 faca:
13        array[i] = array[i + 1]
14    fim_para
15    Retorne resp
```

Custo: Sempre $\Theta(n)$

Mostrar

Em uma fila linear, os elementos válidos estão contíguos do índice 0 até $n - 1$. Basta iterar sequencialmente por esse intervalo imprimindo cada valor, resultando em uma varredura completa proporcional à quantidade de elementos presentes.

```
1 Funcao mostrar():
2     // Passo 1: Imprimir abertura da
3     estrutura
4     Escreva "[ "
5     // Passo 2: Percorrer linearmente do
6     inicio ao fim
7     Para i = 0 ate n - 1 faca:
8         Escreva array[i] + " "
9     fim_para
10    // Passo 3: Imprimir fechamento
11    Escreva "]"
```

Custo: Sempre $\Theta(n)$

Pilha

Inserir no Fim

Na pilha (LIFO), a operação padrão de inserção é o push (empilhar), ocorrendo sempre no topo da estrutura. Como o índice n aponta para a próxima posição disponível logo acima do elemento do topo atual, basta armazenar o novo dado em $\text{array}[n]$ e incrementar n , resultando em tempo constante.

```
1 Funcao empilhar(x):
2     // Passo 1: Validar se a pilha esta cheia
3     Se  $n \geq \text{tamanho}$  entao:
4         Dispare erro "Erro ao inserir!"
5     fim_se
6     // Passo 2: Armazenar no topo e incrementar contador
7      $\text{array}[n] = x$ 
8      $n = n + 1$ 
```

Custo: Sempre $\Theta(1)$

Remover no Fim

A operação padrão de remoção é o pop (desempilhar), que acontece exatamente no topo. Como o topo coincide com o final do vetor no índice $n - 1$, basta decrementar n para liberar a posição e retornar o valor, sem qualquer necessidade de deslocar elementos.

```
1 Funcao desempilhar():
2     // Passo 1: Validar se a pilha esta vazia
3     Se  $n == 0$  entao:
4         Dispare erro "Erro ao remover!"
5     fim_se
6     // Passo 2: Decrementar topo e retornar o elemento
7      $n = n - 1$ 
8     Retorne  $\text{array}[n]$ 
```

Custo: Sempre $\Theta(1)$

Mostrar

Do topo para a base

```
1 Funcao mostrar():
2     // Passo 1: Imprimir abertura da estrutura
3     Escreva "[ "
4     // Passo 2: Percorrer do topo ate a base
5     Para  $i = n - 1$  ate  $0$  passo  $-1$  faca:
6         Escreva  $\text{array}[i] + " "$ 
7     fim_para
8     // Passo 3: Imprimir fechamento
9     Escreva "]"
```

Da base para o topo

```
1 Funcao mostrar():
2     // Passo 1: Imprimir abertura da estrutura
3     Escreva "[ "
4     // Passo 2: Percorrer da base ate o topo
5     Para  $i = 0$  ate  $n - 1$  faca:
6         Escreva  $\text{array}[i] + " "$ 
7     fim_para
8     // Passo 3: Imprimir fechamento
9     Escreva "]"
```

Custo: Sempre $\Theta(n)$

Lista

Inserir

Inserir em uma posição qualquer exige abrir espaço no array: todos os elementos a partir do índice pos até n - 1 devem ser empurrados uma posição para a direita, começando de trás para frente para não sobrescrever dados. No melhor caso inserimos no fim (pos = n, sem deslocamentos), e no pior caso inserimos no início (pos = 0, deslocando todos os n elementos).

```
1 Funcao inserir(x, pos):
2   // Passo 1: Validar se ha espaco e se a posicao e valida
3   Se n >= tamanho ou pos < 0 ou pos > n entao:
4     Dispare erro "Erro ao inserir!"
5   fim_se
6
7   // Passo 2: Deslocar elementos para a direita abrindo espaco
8   Para i = n até pos + 1 passo -1 faca:
9     array[i] = array[i-1]
10  fim_para
11
12  // Passo 3: Inserir elemento na posicao e atualizar tamanho
13  array[pos] = x
14  n = n + 1
```

Custo: Melhor $\Omega(1)$ | Pior $O(n)$

Remover

Remover em uma posição qualquer deixa uma lacuna no vetor. Para manter os dados contíguos, puxamos todos os elementos à direita de pos uma posição para a esquerda (de pos até n - 1). No melhor caso removemos o último elemento (pos = n - 1), onde o laço nem executa e o custo é Omega(1). No pior caso removemos o primeiro elemento (pos = 0), exigindo deslocar todos os elementos restantes, resultando em $O(n)$.

```
1 Funcao remover(pos):
2   // Passo 1: Validar se a lista esta vazia
3   ou posicao e invalida
4   Se n == 0 ou pos < 0 ou pos >= n entao:
5     Dispare erro "Erro ao remover!"
6   fim_se
7   // Passo 2: Salvar o elemento e
8   decrementar tamanho
9   resp = array[pos]
10  n = n - 1
11  // Passo 3: Deslocar elementos para a
12  esquerda cobrindo o espaco
13  Para i = pos ate n - 1 faca:
14    array[i] = array[i + 1]
15  fim_para
16  Retorne resp
```

Custo: Melhor $\Omega(1)$ | Pior $O(n)$

Mostrar

Na lista linear, todos os elementos válidos estão organizados em sequência do índice 0 até n - 1. A exibição simplesmente percorre esse intervalo do início ao fim, visitando cada item exatamente uma vez, o que torna o custo diretamente proporcional ao tamanho atual da lista.

```
1 Funcao mostrar():
2   // Passo 1: Imprimir abertura da
3   estrutura
4   Escreva "[ "
5   // Passo 2: Percorrer linearmente do
6   inicio ao fim
7   Para i = 0 ate n - 1 faca:
8     Escreva array[i] + " "
9   fim_para
10  // Passo 3: Imprimir fechamento
11  Escreva "]"
```

Custo: Sempre $\Theta(n)$

Fila Circular

Inserir

Em uma fila circular, o ponteiro ultimo aponta para a próxima posição livre. Usamos a operação mod tamanho para fazer o índice dar a volta ao início do vetor quando chega ao fim, reservando uma posição vazia para diferenciar a fila cheia da fila vazia

```
1 Funcao inserir(x):
2     // Passo 1: Validar se a fila circular esta cheia
3     Se ((ultimo + 1) mod tamanho) == primeiro entao:
4         Dispare erro "Erro ao inserir!"
5     fim_se
6     // Passo 2: Inserir elemento no fim e atualizar ponteiro
7     array[ultimo] = x
8     ultimo = (ultimo + 1) mod tamanho
```

Custo: Sempre $\Theta(1)$

Remover

Em uma fila, a remoção sempre ocorre no início (primeiro). Se primeiro == ultimo, a estrutura está vazia. Ao retirar o elemento, apenas avançamos o índice primeiro com (primeiro + 1) mod tamanho, liberando a posição logicamente sem precisar deslocar nenhum dado na memória.

```
1 Funcao remover():
2     // Passo 1: Validar se a fila circular esta vazia
3     Se primeiro == ultimo entao:
4         Dispare erro "Erro ao remover!"
5     fim_se
6     // Passo 2: Resgatar o elemento da frente
7     resp = array[primeiro]
8     // Passo 3: Avançar o ponteiro inicial de forma circular
9     primeiro = (primeiro + 1) mod tamanho
10    Retorne resp
```

Custo: Sempre $\Theta(1)$

Mostrar

Como a estrutura é circular, a iteração começa no índice primeiro e avança com (i + 1) mod tamanho até encontrar ultimo. Cada elemento presente é visitado exatamente uma vez.

```
1 Funcao mostrar():
2     // Passo 1: Imprimir abertura da estrutura
3     Escreva "["
4     // Passo 2: Percorrer os elementos do primeiro ate o ultimo de forma circular
5     Para i = primeiro ate (i != ultimo) passo i = (i + 1) mod tamanho faca:
6         Escreva array[i] + " "
7     fim_para
8     // Passo 3: Imprimir fechamento
9     Escreva "]"
```

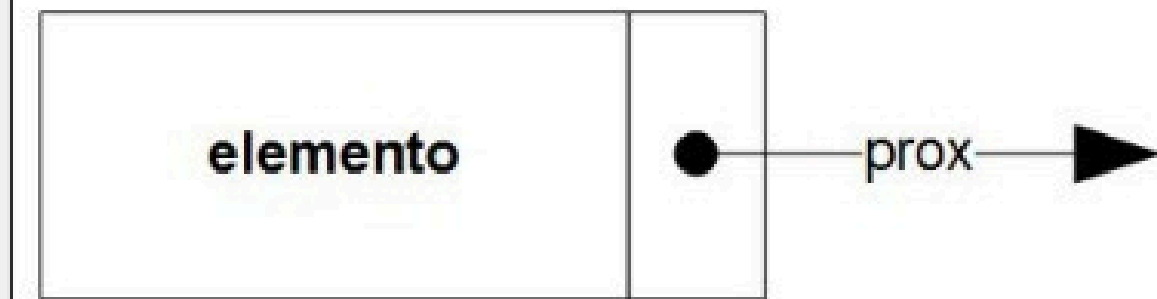
```
1 Funcao mostrarReverso():
2     // Passo 1: Imprimir abertura da estrutura
3     Escreva "["
4     // Passo 2: Definir a parada (uma posicao
5     // antes do 'primeiro')
6     limite = (primeiro - 1 + tamanho) mod tamanho
7     // Passo 3: Percorrer de tras pra frente
8     // ate esbarrar no limite
9     Para i = (ultimo - 1 + tamanho) mod
10 tamanho ate (i != limite) passo i = (i - 1 + tamanho) mod tamanho faca:
11     Escreva array[i] + " "
12 fim_para
13 // Passo 4: Imprimir fechamento
14 Escreva "]"
```

Custo: Sempre $\Theta(n)$

Classe Celula

Essa será a classe célula utilizada em todas as estruturas **encadeadas** (estruturas que podem ou não utilizar célula cabeça para implementação)

```
class Celula {  
    public int elemento;  
    public Celula prox;  
    public Celula() {  
        this(0);  
    }  
    public Celula (int x) {  
        this.elemento = x;  
        this.prox = null;  
    }  
}
```



Lista Simples

INSERIR NO INICIO

INSERIR NO FIM

INSERIR

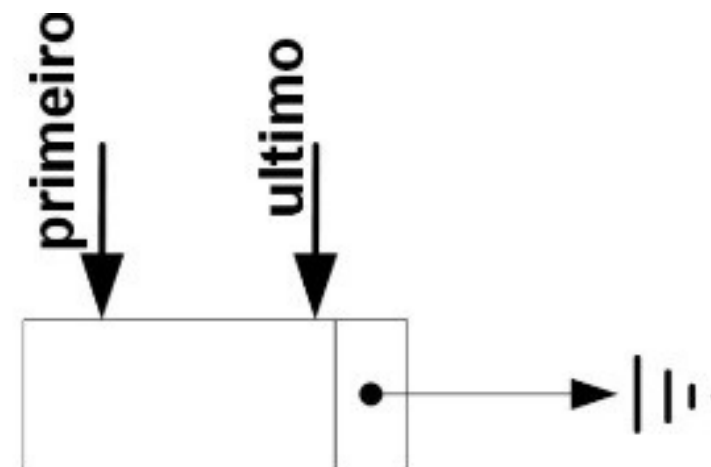
REMOVER NO INÍCIO

REMOVER NO FIM

REMOVER

Lista Simples

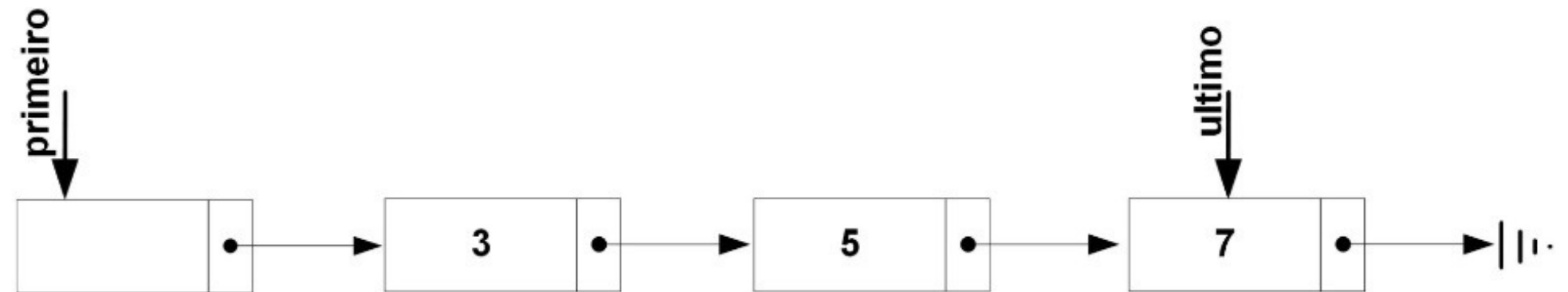
```
class Lista {  
    private Celula primeiro, ultimo;  
    public Lista () {  
        primeiro = new Celula();  
        ultimo = primeiro;  
    }  
    public void inserirInicio(int x) { ... }  
    public void inserirFim(int x) { ... }  
    public int removerInicio() { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
    public void mostrar() { ... }  
}
```



Lista Simples

Os métodos Inserir Fim e Remover Início são iguais aos métodos da Fila.

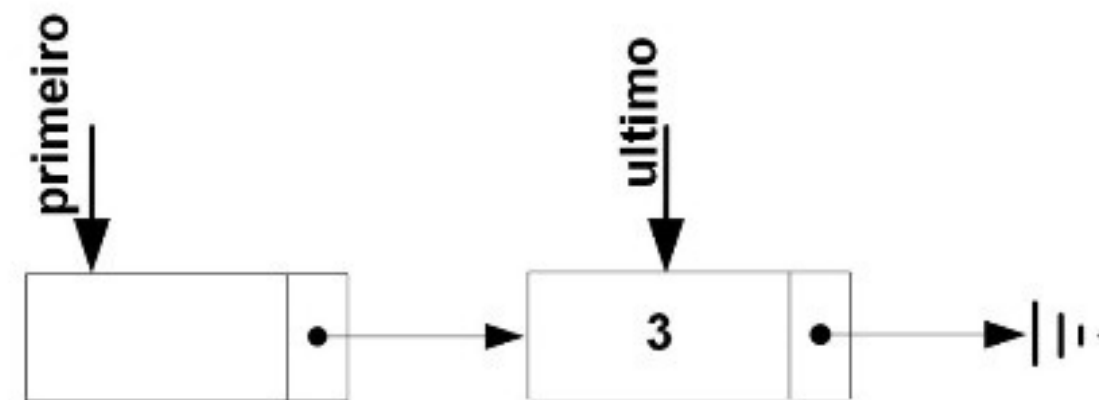
```
class Lista {  
    private Celula primeiro, ultimo;  
    public Lista () {  
        primeiro = new Celula();  
        ultimo = primeiro;  
    }  
    public void inserirInicio(int x) { ... }  
    public void inserirFim(int x) { ... }  
    public int removerInicio() { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
    public void mostrar() { ... }  
}
```



Lista Simples Inserir fim

Os métodos Inserir Fim e Remover Início são iguais aos métodos da Fila.

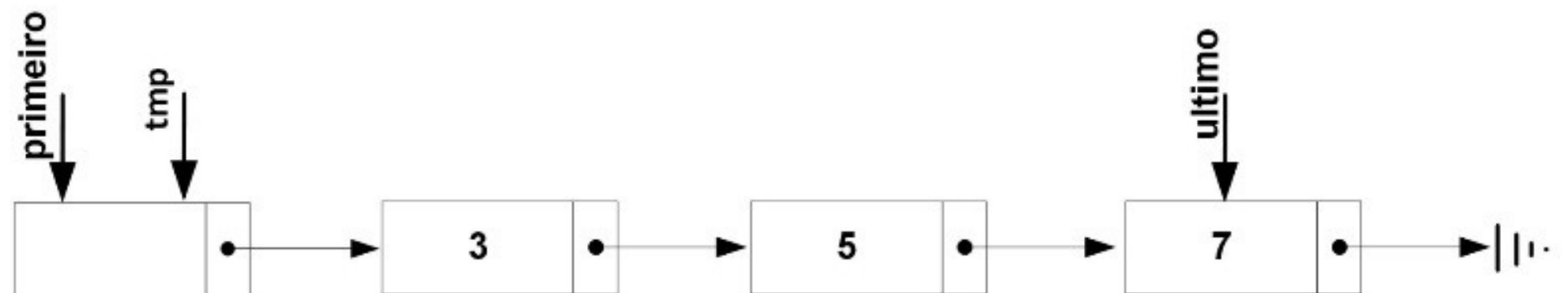
```
public void inserir(int x) { //Inserir(3)  
    ultimo.prox = new Celula(x);  
    ultimo = ultimo.prox;  
}
```



Lista Simples remover início

Os métodos Inserir Fim e Remover Início são iguais aos métodos da Fila.

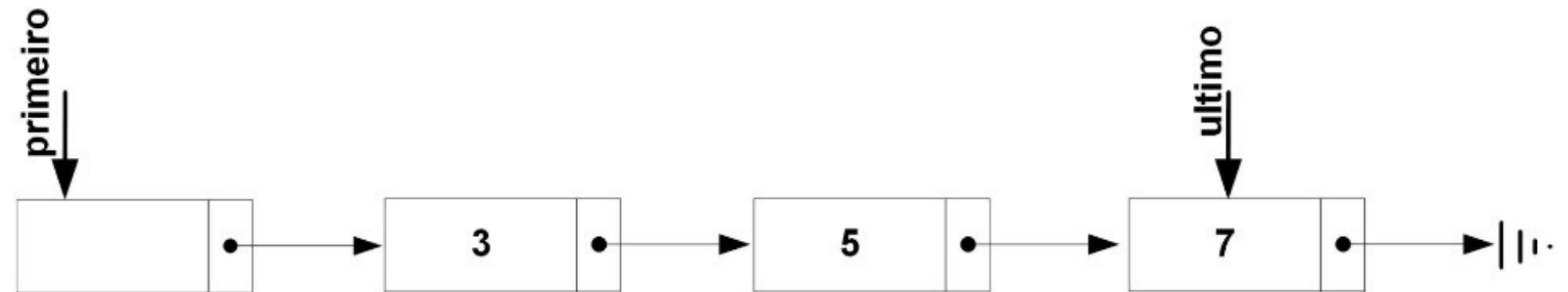
```
public int remover() throws Exception{  
    if (primeiro == ultimo)  
        throw new Exception("Erro!");  
    Celula tmp = primeiro;  
    primeiro = primeiro.prox;  
    int elemento = primeiro.elemento;  
    tmp.prox = null;  
    tmp = null;  
    return elemento;  
}
```



Lista Simples

O método Mostrar é igual ao da Fila/Pilha

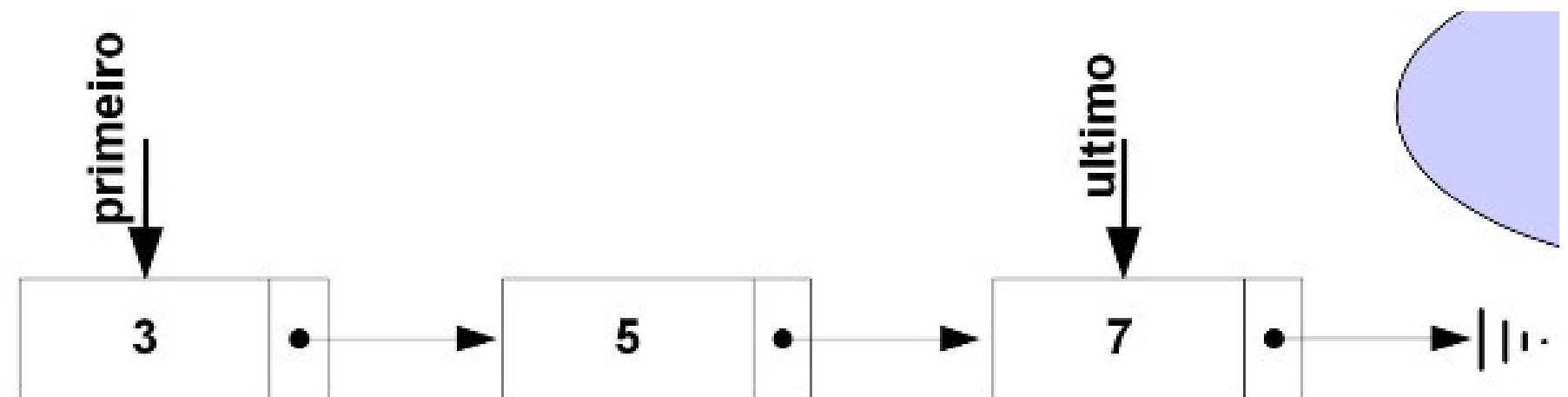
```
class Lista {  
    private Celula primeiro, ultimo;  
    public Lista () {  
        primeiro = new Celula();  
        ultimo = primeiro;  
    }  
    public void inserirInicio(int x) { ... }  
    public void inserirFim(int x) { ... }  
    public int removerInicio() { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
    public void mostrar() { ... }  
}
```



Lista Simples

O método Mostrar é igual ao da Fila/Pilha

```
    * Mostra os elementos separados por espacos.  
    */  
public void mostrar() {  
    System.out.print("[ ");  
  
    for(Celula i = primeiro.prox; i != null; i = i.prox) {  
        System.out.print(i.elemento + " ");  
    }  
  
    System.out.println("] ");  
}
```



Lista Simples Inserir Início

```
class Lista {
```

```
...
```

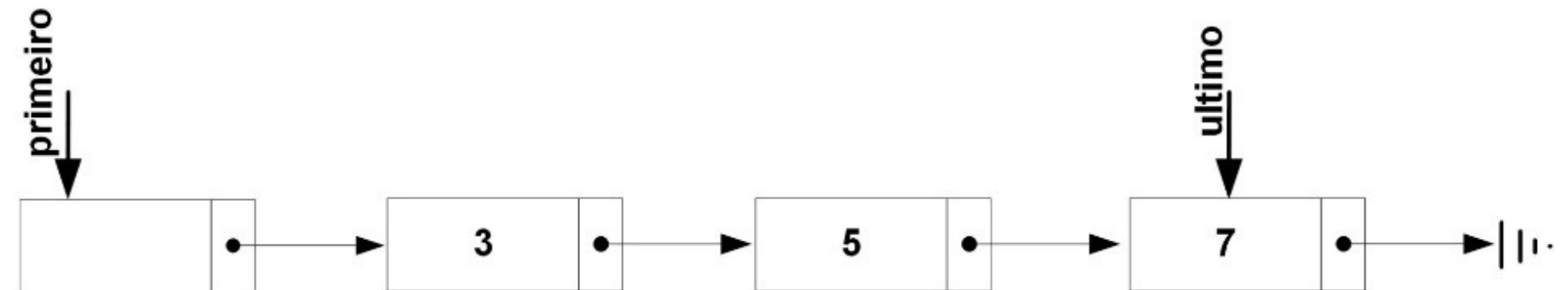
```
public void inserirInicio(int x) { ... }
```

```
public int removerFim() { ... }
```

```
public void inserir(int x, int pos) { ... }
```

```
public int remover(int pos) { ... }
```

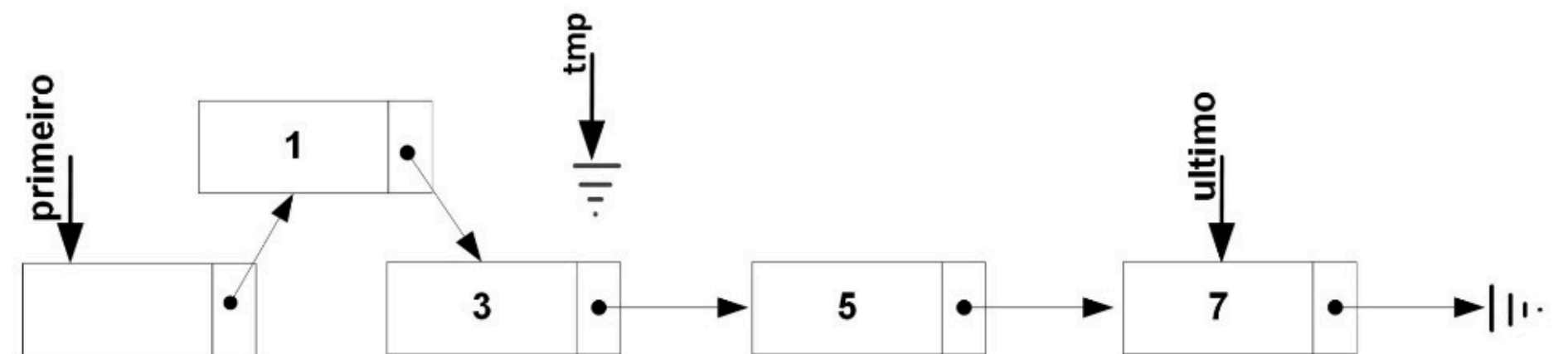
```
}
```



Lista Simples Inserir Início

//inserirInicio(1)

```
public void inserirInicio(int x) {  
    Celula tmp = new Celula(x);  
    tmp.prox = primeiro.prox;  
    primeiro.prox = tmp;  
    if (primeiro == ultimo) {  
        ultimo = tmp;  
    }  
    tmp = null;  
}
```



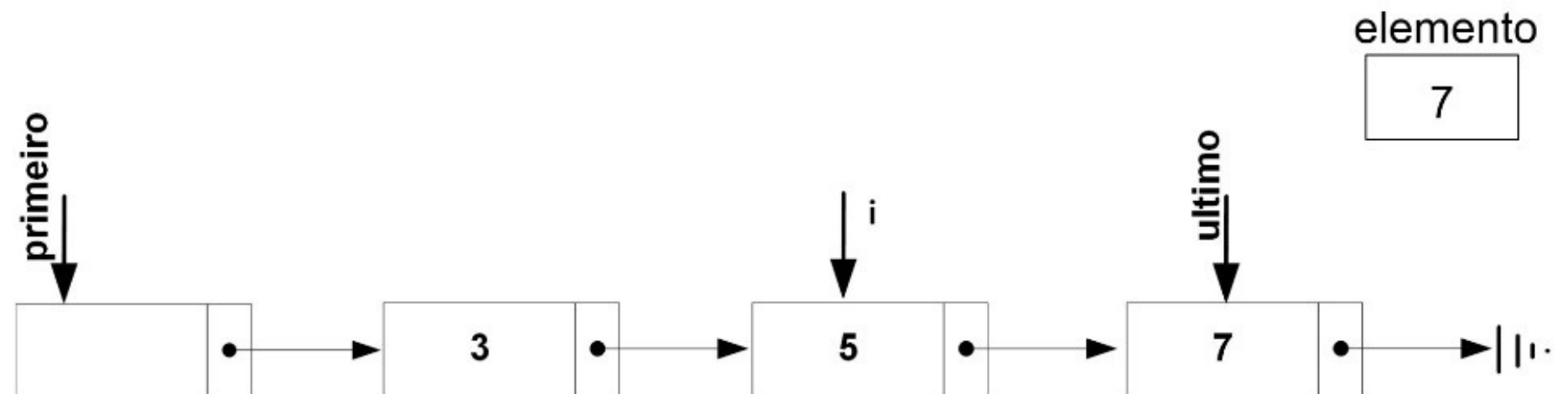
Lista Simples Remover Fim

```
class Lista {  
    ...  
    public void inserirInicio(int x) { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
}
```



Lista Simples Remover Fim

```
public int removerFim() throws Exception {  
    if (primeiro == ultimo)  
        throw new Exception("Erro!");  
    Celula i;  
    for(i = primeiro; i.prox != ultimo; i = i.prox);  
    int elemento = ultimo.elemento;  
    ultimo = i;  
    i = ultimo.prox = null;  
    return elemento;  
}
```



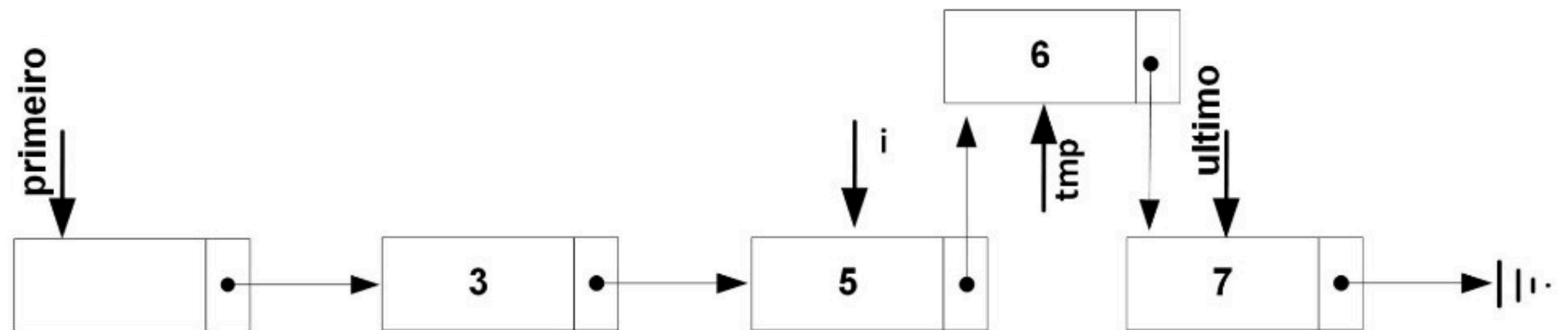
Lista Simples

Inserir

```
class Lista {  
    ...  
    public void inserirInicio(int x) { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
}
```

Lista Simples Inserir

```
public void inserir(int x, int pos) throws Exception {    //Inserir(6, 2)
    int tamanho = tamanho();
    if (pos < 0 || pos > tamanho){ throw new Exception("Erro!");
    } else if (pos == 0){        inserirInicio(x);
    } else if (pos == tamanho){  inserirFim(x);
    } else {
        Celula i = primeiro;
        for(int j = 0; j < pos; j++, i = i.prox);
        Celula tmp = new Celula(x);
        tmp.prox = i.prox;
        i.prox = tmp;
        tmp = i = null;
    }
}
```



Lista Simples Remover

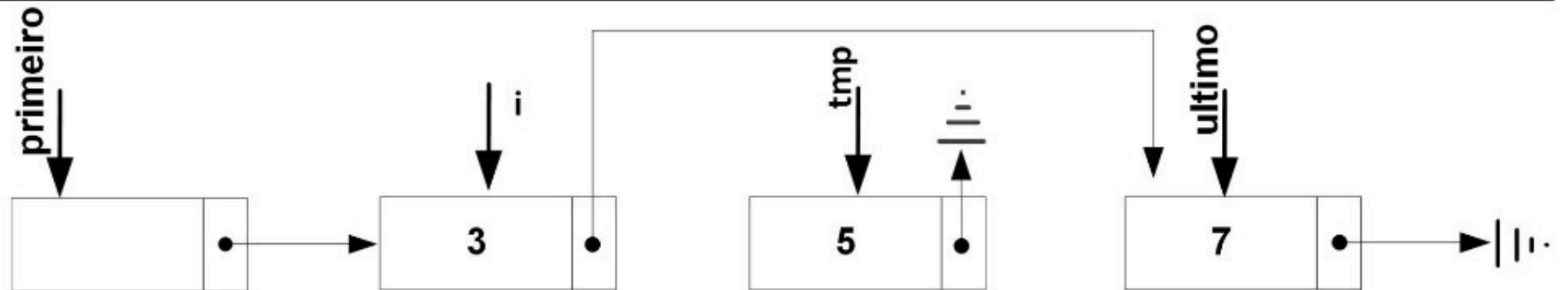
```
class Lista {  
    ...  
    public void inserirInicio(int x) { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
}
```

Lista Simples Remover

```
public int remover(int pos ) throws Exception {           //remover(1)
    int elemento, tamanho = tamanho();
    if (primeiro == ultimo || pos < 0 || pos >= tamanho){ throw new Exception("Erro!");
    } else if (pos == 0) {                                elemento = removerInicio();
    } else if (pos == tamanho - 1){                       elemento = removerFim();
    } else {
        Celula i = primeiro;
        for(int j = 0; j < pos; j++, i = i.prox);
        Celula tmp = i.prox;
        elemento = tmp.elemento;    i.prox = tmp.prox;
        tmp.prox = null;           i = tmp = null;
    }
    return elemento;
}
```

elemento

5



Lista Dupla

INSERIR NO INÍCIO

INSERIR NO FIM

INSERIR

REMOVER NO INÍCIO

REMOVER NO FIM

REMOVER

Lista Dupla Classe CelulaDupla

Similar a classe **Celula** antes vista, o principal diferencial é a criação de mais um ponteiro.

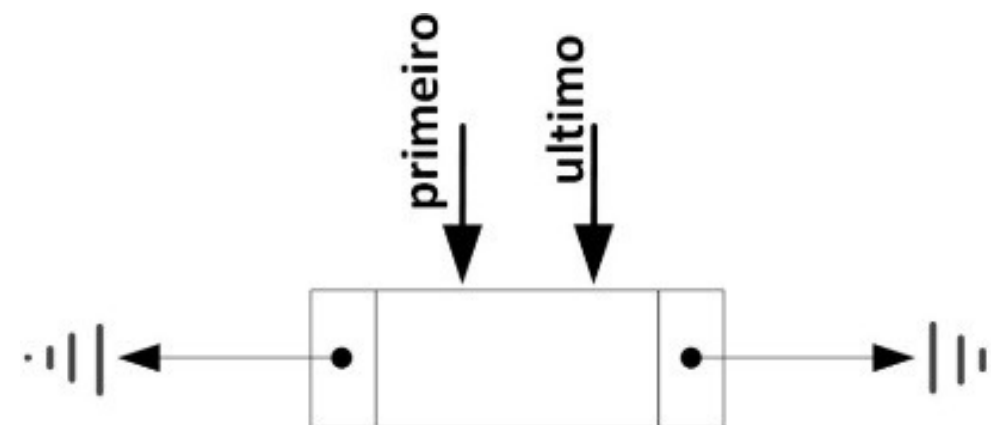
```
class CelulaDupla {  
    public int elemento;  
    public CelulaDupla prox, ant;  
    public CelulaDupla () {  
        this(0);  
    }  
    public CelulaDupla (int x) {  
        this.elemento = x;  
        this.prox = this.ant = null;  
    }  
}
```



Lista Dupla

Similar a Lista Simples, contudo considerando o ponteiro **ant**.

```
class ListaDupla {  
    private CelulaDupla primeiro, ultimo;  
    public ListaDupla () {  
        primeiro = new CelulaDupla();  
        ultimo = primeiro;  
    }  
    public void inserirInicio(int x) { ... }  
    public void inserirFim(int x) { ... }  
    public int removerInicio() { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
    public void mostrar() { ... }  
}
```



Lista Dupla

Inserir Início

```
class ListaDupla {  
    private CelulaDupla primeiro, ultimo;  
    public ListaDupla () {  
        primeiro = new CelulaDupla();  
        ultimo = primeiro;  
    }  
    public void inserirInicio(int x) { ... }  
    public void inserirFim(int x) { ... }  
    public int removerInicio() { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
    public void mostrar() { ... }  
}
```

Lista Dupla Inserir Início

//LISTA DUPLA

```
public void inserirInicio(int x) {  
    CelulaDupla tmp = new CelulaDupla(x);  
    tmp.ant = primeiro;  
    tmp.prox = primeiro.prox;  
    primeiro.prox = tmp;  
    if (primeiro == ultimo) {  
        ultimo = tmp;  
    } else {  
        tmp.prox.ant = tmp;  
    }  
    tmp = null;  
}
```

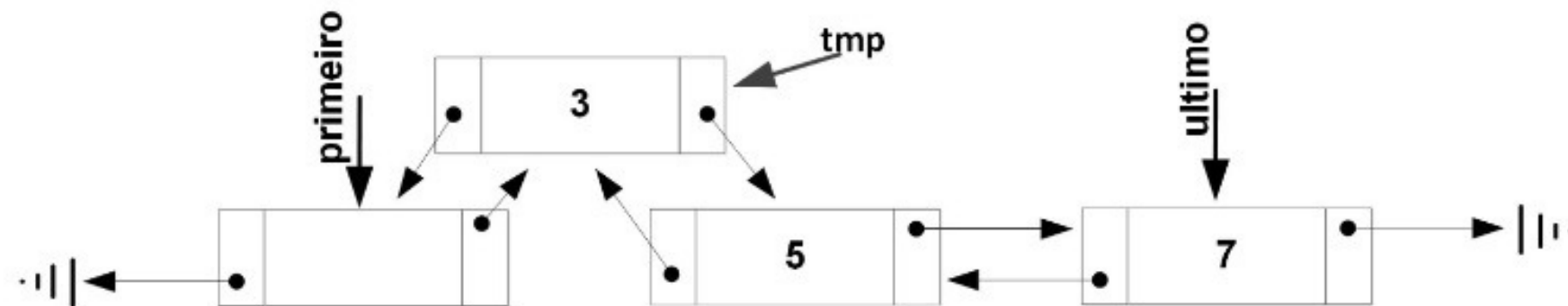
//LISTA SIMPLES

```
public void inserirInicio(int x) {  
    Celula tmp = new Celula(x);  
  
    tmp.prox = primeiro.prox;  
    primeiro.prox = tmp;  
    if (primeiro == ultimo) {  
        ultimo = tmp;  
    }  
    tmp = null;  
}
```

Lista Dupla Inserir Início

//Inserindo o 3 no início

```
public void inserirInicio(int x) {  
    CelulaDupla tmp = new CelulaDupla(x);  
    tmp.ant = primeiro;  
    tmp.prox = primeiro.prox;  
    primeiro.prox = tmp;  
    if (primeiro == ultimo) {  
        ultimo = tmp;  
    } else {  
        tmp.prox.ant = tmp;  
    }  
    tmp = null;  
}
```



Lista Dupla Inserir Fim

```
class ListaDupla {  
    private CelulaDupla primeiro, ultimo;  
    public ListaDupla () {  
        primeiro = new CelulaDupla();  
        ultimo = primeiro;  
    }  
    public void inserirInicio(int x) { ... }  
    public void inserirFim(int x) { ... }  
    public int removerInicio() { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
    public void mostrar() { ... }  
}
```

Lista Dupla Inserir Fim

//LISTA DUPLA

```
public void inserirFim(int x) {  
    ultimo.prox = new CelulaDupla(x);  
    ultimo.prox.ant = ultimo;  
    ultimo = ultimo.prox;  
}
```

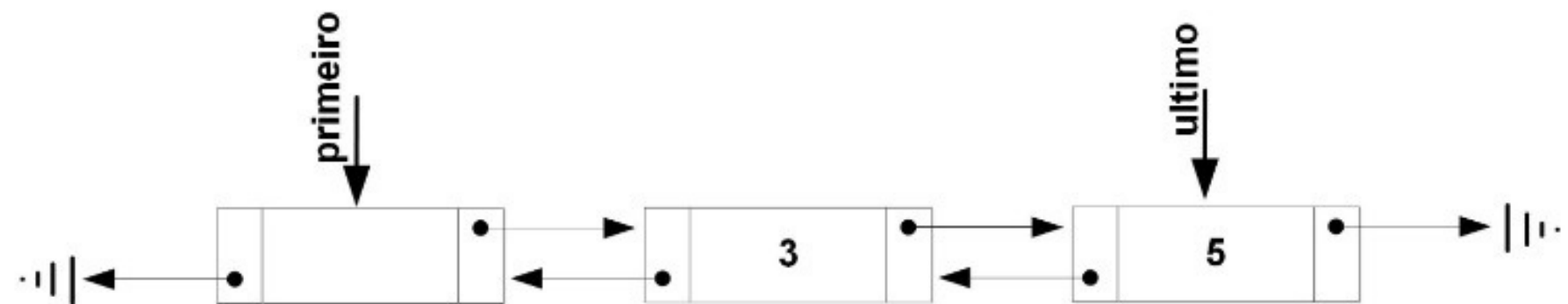
//LISTA SIMPLES

```
public void inserirFim(int x) {  
    ultimo.prox = new Celula(x);  
  
    ultimo = ultimo.prox;  
}
```

Lista Dupla Inserir Fim

//LISTA DUPLA

```
public void inserirFim(int x) {  
    ultimo.prox = new CelulaDupla(x);  
    ultimo.prox.ant = ultimo;  
    ultimo = ultimo.prox;  
}
```



Lista Dupla Remover Início

//LISTA DUPLA

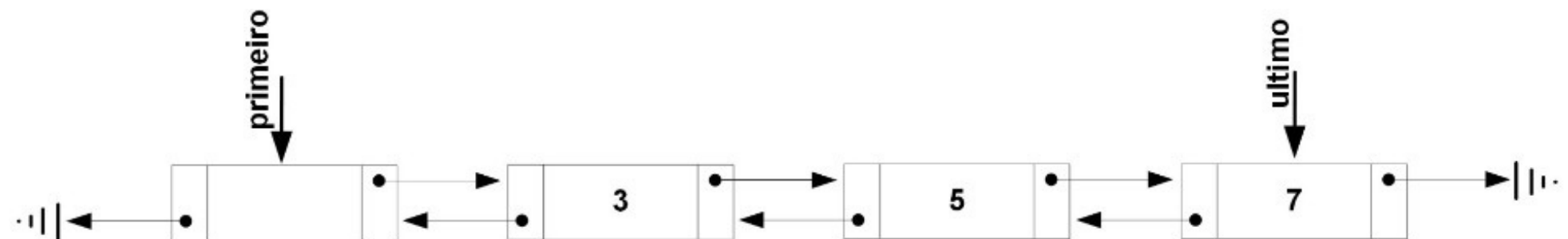
```
public int removerInicio() throws Exception {  
    if (primeiro == ultimo)  
        throw new Exception("Erro!");  
    CelulaDupla tmp = primeiro;  
    primeiro = primeiro.prox;  
    int elemento = primeiro.elemento;  
    tmp.prox = primeiro.ant = null;  
    tmp = null;  
    return elemento;  
}
```

//LISTA SIMPLES

```
public int removerInicio() throws Exception {  
    if (primeiro == ultimo)  
        throw new Exception("Erro!");  
    Celula tmp = primeiro;  
    primeiro = primeiro.prox;  
    int elemento = primeiro.elemento;  
    tmp.prox = null;  
    tmp = null;  
    return elemento;  
}
```

Lista Dupla Remover Início

```
//LISTA DUPLA  
public int removerInicio() throws Exception {  
    if (primeiro == ultimo)  
        throw new Exception("Erro!");  
    CelulaDupla tmp = primeiro;  
    primeiro = primeiro.prox;  
    int elemento = primeiro.elemento;  
    tmp.prox = primeiro.ant = null;  
    tmp = null;  
    return elemento;  
}
```



Lista Dupla Remover Fim

```
class ListaDupla {  
    private CelulaDupla primeiro, ultimo;  
    public ListaDupla () {  
        primeiro = new CelulaDupla();  
        ultimo = primeiro;  
    }  
    public void inserirInicio(int x) { ... }  
    public void inserirFim(int x) { ... }  
    public int removerInicio() { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
    public void mostrar() { ... }  
}
```

Lista Dupla Remover Fim

//LISTA DUPLA

```
public int removerFim() throws Exception {  
    if (primeiro == ultimo)  
        throw new Exception("Erro!");  
  
    int elemento = ultimo.elemento;  
    ultimo = ultimo.ant;  
    ultimo.prox.ant = null;  
    ultimo.prox = null;  
    return elemento;  
}
```

//LISTA SIMPLES

```
public int removerFim() throws Exception {  
    if (primeiro == ultimo)  
        throw new Exception("Erro!");  
    Celula i;  
    for(i = primeiro; i.prox != ultimo; i = i.prox);  
    int elemento = ultimo.elemento;  
    ultimo = i;  
  
    i = ultimo.prox = null;  
    return elemento;  
}
```

Lista Dupla Remover Fim

//LISTA DUPLA

```
public int removerFim() throws Exception {
```

```
    if (primeiro == ultimo)
```

```
        throw new Exception("Erro!");
```

```
    int elemento = ultimo.elemento;
```

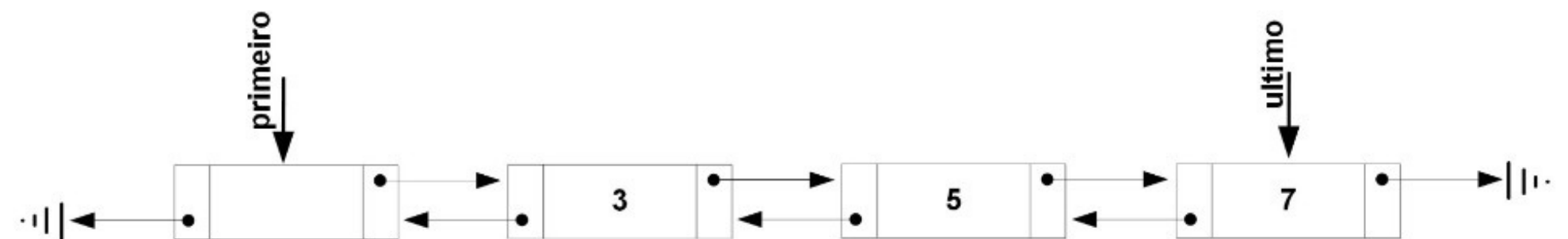
```
    ultimo = ultimo.ant;
```

```
    ultimo.prox.ant = null;
```

```
    ultimo.prox = null;
```

```
    return elemento;
```

```
}
```



Lista Dupla

Inserir

```
class ListaDupla {  
    private CelulaDupla primeiro, ultimo;  
    public ListaDupla () {  
        primeiro = new CelulaDupla();  
        ultimo = primeiro;  
    }  
    public void inserirInicio(int x) { ... }  
    public void inserirFim(int x) { ... }  
    public int removerInicio() { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
    public void mostrar() { ... }  
}
```

Lista Dupla Inserir

//LISTA DUPLA

```
public void inserir(int x, int pos) throws
Exception {
    int tamanho = tamanho();
    if (pos < 0 || pos > tamanho){
        throw new Exception("Erro!");
    } else if (pos == 0){ inserirInicio(x);
    } else if (pos == tamanho){ inserirFim(x);
    } else {
        CelulaDupla i = primeiro;
        for (int j = 0; j < pos; j++, i = i.prox);

        CelulaDupla tmp = new CelulaDupla(x);
        tmp.ant = i;
        tmp.prox = i.prox;
        tmp.ant.prox = tmp.prox.ant = tmp;
        tmp = i = null;
    }
}
```

//LISTA SIMPLES

```
public void inserir(int x, int pos) throws
Exception {
    int tamanho = tamanho();
    if (pos < 0 || pos > tamanho){
        throw new Exception("Erro!");
    } else if (pos == 0){ inserirInicio(x);
    } else if (pos == tamanho){ inserirFim(x);
    } else {
        Celula i = primeiro;
        for (int j = 0; j < pos; j++, i = i.prox);

        Celula tmp = new Celula(x);

        tmp.prox = i.prox;
        i.prox = tmp;
        tmp = i = null;
    }
}
```

Lista Dupla Remover

```
class ListaDupla {  
    private CelulaDupla primeiro, ultimo;  
    public ListaDupla () {  
        primeiro = new CelulaDupla();  
        ultimo = primeiro;  
    }  
    public void inserirInicio(int x) { ... }  
    public void inserirFim(int x) { ... }  
    public int removerInicio() { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
    public void mostrar() { ... }  
}
```

Lista Dupla Remover

//LISTA DUPLA

```
public int remover(int pos) throws Exception {
    int elemento, tamanho = tamanho();
    if (primeiro == ultimo){
        throw new Exception("Erro!");
    } else if (pos < 0 || pos >= tamanho){
        throw new Exception("Erro!");
    } else if (pos == 0){
        elemento = removerInicio();
    } else if (pos == tamanho - 1){
        elemento = removerFim();
    } else {
        CelulaDupla i = primeiro.prox;
        for (int j = 0; j < pos; j++, i = i.prox);
        i.ant.prox = i.prox;
        i.prox.ant = i.ant;
        elemento = i.elemento;
        i.prox = i.ant = null;
        i = null;
    }
    return elemento;
}
```

//LISTA SIMPLES

```
public int remover(int pos) throws Exception {
    int elemento, tamanho = tamanho();
    if (primeiro == ultimo){
        throw new Exception("Erro!");
    } else if (pos < 0 || pos >= tamanho){
        throw new Exception("Erro!");
    } else if (pos == 0){
        elemento = removerInicio();
    } else if (pos == tamanho - 1){
        elemento = removerFim();
    } else {
        Celula i = primeiro;
        for (int j = 0; j < pos; j++, i = i.prox);
        Celula tmp = i.prox;
        elemento = tmp.elemento;
        i.prox = tmp.prox;
        tmp.prox = null;
        i = tmp = null;
    }
    return elemento;
}
```

Lista Dupla Mostrar

```
class ListaDupla {  
    private CelulaDupla primeiro, ultimo;  
    public ListaDupla () {  
        primeiro = new CelulaDupla();  
        ultimo = primeiro;  
    }  
    public void inserirInicio(int x) { ... }  
    public void inserirFim(int x) { ... }  
    public int removerInicio() { ... }  
    public int removerFim() { ... }  
    public void inserir(int x, int pos) { ... }  
    public int remover(int pos) { ... }  
    public void mostrar() { ... }  
}
```

